



IE3142: DevOps Security

3rd Year, 1st Semester

Assignment

Building and Securing a DevSecOps Pipeline

Group No 12

Name	RegNO
Vitharana I.N.S.	IT24102980
Navoda M.M.	IT24102054
Rasanajan S.P.	IT24102936
S W Wickramathilake	IT24103594

Table of Contents

Executive Summary and System Overview	1
System Architecture and Trust Boundaries Diagram.	2
Threat Modelling & Risk Assessment (STRIDE)	3
1 IDOR	5
2 SQL Injection	7
3 Stored XSS	9
4 Security Misconfiguration.....	12
SAST Validations (Semgrep)	14
1 Secrets Scanning – Gitleaks	15
2 Dependency Scanning – npm audit.....	15
3 Static Application Security Testing – Semgrep	15
Secrets Management Approach	18
Industry Trends & Case Study Analysis.....	19
Reflection & Future Improvements	20
Individual Contribution & AI Usage Disclosure	21
References	23

Executive Summary and System Overview

The main goal of this project is to develop a DevSecOps pipeline for an open-source web application, where security is integrated into the software development and deployment process. The project focuses on identifying security risks early, continuously testing the application, and improving the overall security of the system throughout its lifecycle.

For this project, OWASP Juice Shop was selected as the open-source application. OWASP Juice Shop is a deliberately insecure web application commonly used for security testing and learning. It is built using a Node.js backend, Angular-based frontend, and database components, making it suitable for demonstrating different security testing and DevSecOps practices.

The application is containerized using Docker and Docker Compose. The backend handles application logic and communicates with the local SQLite database using SQL queries via the Sequelize ORM. The frontend provides the user interface and communicates with the backend through a REST API using HTTP/JSON. The backend handles application logic and communicates with the MySQL database using SQL queries. This separation makes the system easier to manage, deploy, test, and secure.

The architecture also defines different trust boundaries. External users access the application through HTTPS, while the frontend and backend operate within the internal application network. The database is placed in a separate restricted network area and is intended to be accessed only by the backend service. This architecture provides a clear foundation for applying security controls and DevSecOps practices throughout the project.

System Architecture and Trust Boundaries Diagram.

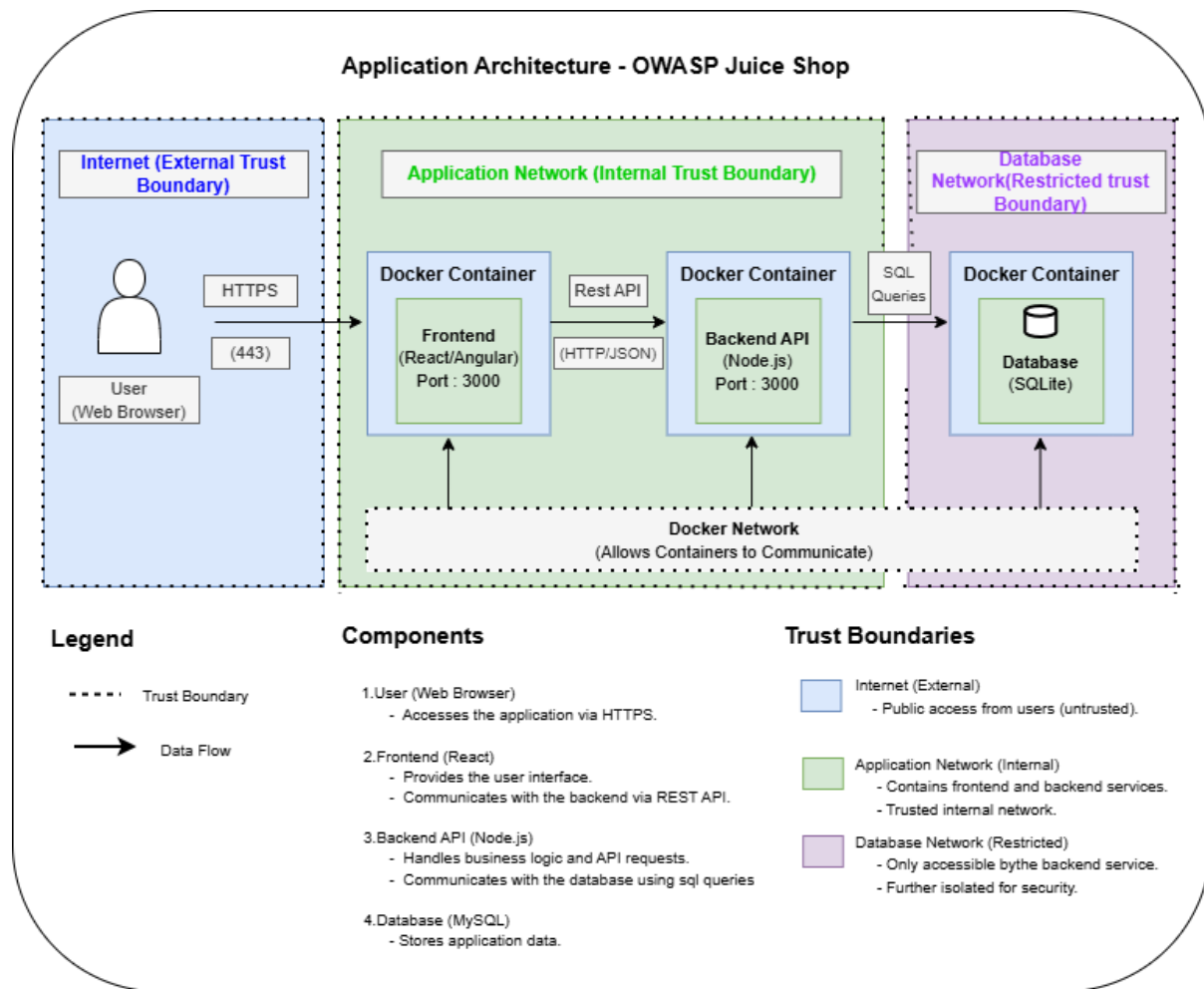


Fig.1. System Architecture and Trust Boundaries Diagram

Threat Modelling & Risk Assessment (STRIDE)

For this project, we used the STRIDE threat modelling methodology to identify security threats within the OWASP Juice Shop architecture. STRIDE helps us analyse the system from different security perspectives, including Spoofing, Tampering, and Information Disclosure. Each identified threat was mapped to a specific component of the application, rated by both likelihood and impact, and linked to the exact file or pipeline control responsible for mitigating it.

Threat	STRIDE	Component	Description	Likelihood	Impact	Justification of Rating	Specific Control & Location (Codebase/Pipeline)
SQL Injection	Tampering	Backend API	Malicious SQL input manipulates database queries.	High	High	Likelihood is High because Juice Shop has documented, easily exploitable login flaws. Impact is High because successful exploitation grants full unauthorized database access.	Parameterized queries implemented in /routes/login.ts and /data/datacreator.ts using Sequelize ORM to prevent query manipulation.
Stored XSS	Tampering	Frontend	Malicious script stored and executed in users' browsers.	High	Medium	Likelihood is High due to user-generated content (reviews). Impact is Medium because it steals session cookies but does not directly compromise the backend database.	Output encoding/DOM sanitization enforced in Angular review.component.ts using DomSanitizer, plus CSP headers in server.ts.

Threat	STRIDE	Component	Description	Likelihood	Impact	Justification of Rating	Specific Control & Location (Codebase/Pipeline)
Authentication Bypass	Spoofing	Backend	Unauthorized users gain access without valid credentials.	Medium	High	Likelihood is Medium as it requires specific knowledge of JWT flaws. Impact is High because it grants full account takeover and admin privileges.	Strong session validation enforced in the JWT middleware located at /lib/insecurity.ts, using a strong secret injected via GitHub Actions Secrets.
Sensitive Data Exposure	Info Disclosure	Database / API	Sensitive info exposed through insecure responses or misconfiguration.	Medium	High	Likelihood is Medium as it requires API probing. Impact is High due to GDPR/privacy violations from exposing customer emails, addresses, and order history.	Field filtering in /routes/user.ts to exclude passwords; bcrypt hashing in /models/user.ts; and HTTPS enforced at the reverse proxy/Docker network level.

These threats were mapped to security controls at both the application code level and CI/CD pipeline level, as shown in the table above. For example, parameterized queries and input validation reduce SQL injection risk, while DOM sanitization and CSP headers address stored XSS. Within the CI/CD pipeline, automated security checks including static code analysis, dependency scanning, and container scanning identify these types of vulnerabilities before deployment, ensuring that security is not treated as a final step but is incorporated throughout the development, testing, and deployment lifecycle.

Secure Coding Evidence

1 IDOR

The Vulnerability:

IDOR occurs when an authenticated user can access another user's resources by changing an object ID without proper authorization checking.

The Exploit:

The vulnerability was tested against the Juice Shop basket API by changing the basket ID in the request. Before applying the fix, the request for another basket returned the basket contents successfully, including product information and the associated user ID. This demonstrated that the server was returning another basket's data without properly verifying ownership.

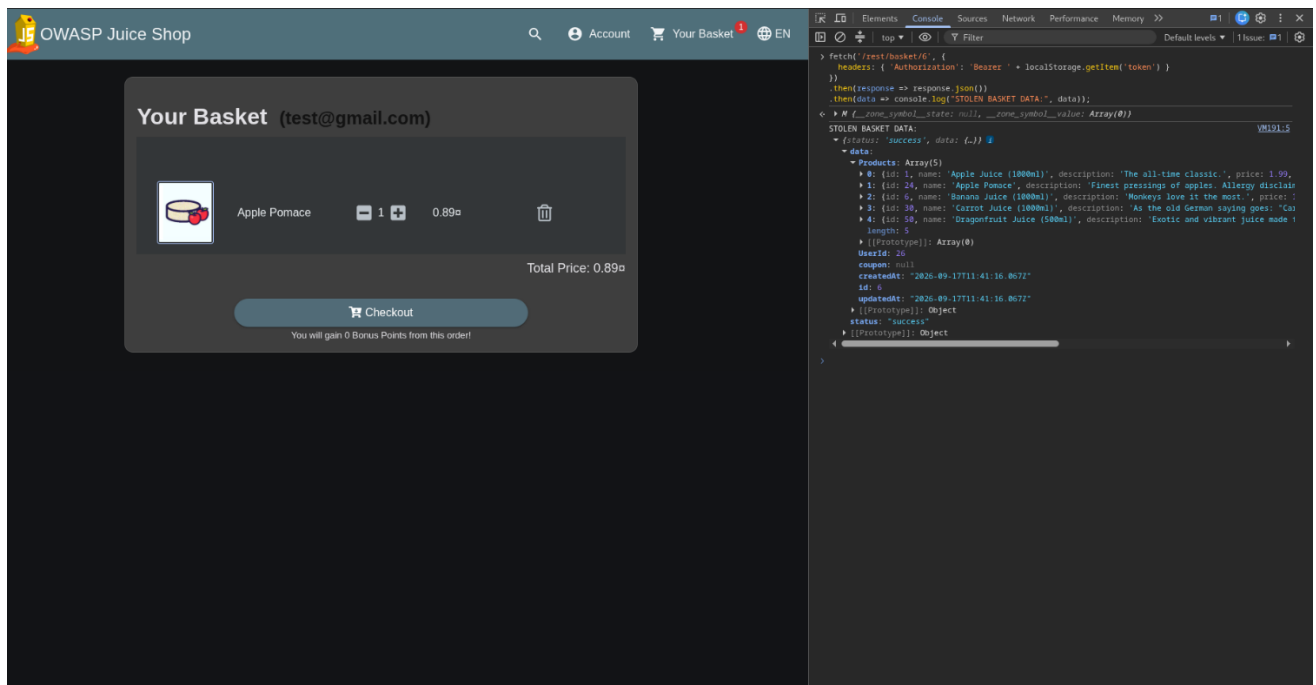


Fig.2.IDOR vulnerability successfully exploited before the fix

The Fix:

The authorization check was strengthened in the routes/basket.ts file. The database query was changed to verify both the requested basket ID and the authenticated user's ID using UserId: user?.data?.id. If the basket does not belong to the authenticated user, the application returns an Access denied (403) response. The Juice Shop source uses routes/basket.ts for the basket retrieval endpoint.

```

import { type Request, type Response, type NextFunction } from 'express'
import { ProductModel } from '../models/product'
import { BasketModel } from '../models/basket'
import * as challengeUtils from '../lib/challengeUtils'

import * as utils from '../lib/utils'
import * as security from '../lib/insecurity'
import { challenges } from '../data/datacache'

export function retrieveBasket () {
  return async (req: Request, res: Response, next: NextFunction) => {
    try {
      const id = req.params.id
      const user = security.authenticatedUsers.from(req)
      const basket = await BasketModel.findOne({
        where: { id, UserId: user?.data?.id },
        include: [{ model: ProductModel, paranoid: false, as: 'Products' }]
      })

      if (!basket) {
        return res.status(403).json({ error: 'Access denied' })
      }

      /* jshint eqeqeq:false */
      challengeUtils.solveIf(challenges.basketAccessChallenge, () => {
        return user && id && id !== 'undefined' && id !== 'null' && id !== 'NaN' && user.bid && user?.bid !== parseInt(id, 10)
      })

      if (((basket?.Products) !== null) && basket.Products.length > 0) {
        for (let i = 0; i < basket.Products.length; i++) {
          basket.Products[i].name = req.__(basket.Products[i].name)
        }
      }

      res.json(utils.queryResultToJson(basket))
    } catch (error) {
      next(error)
    }
  }
}

```

Fig.2.1.IDOR fix implemented in routes/basket.ts

The Validation:

After applying the fix, the same IDOR attack was repeated. Instead of returning the requested basket information, the server rejected the unauthorized request with 403 Forbidden and displayed “Access denied” in the browser console.

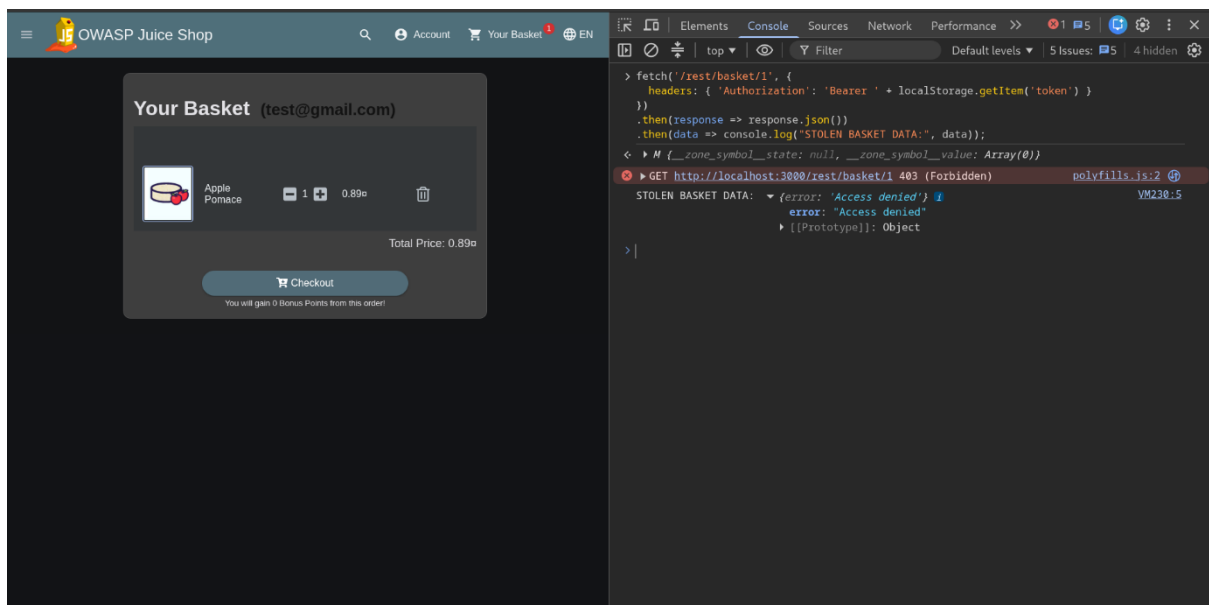


Fig.2.3.IDOR attack blocked after the fix

2 SQL Injection

The Vulnerability:

SQL Injection occurs when user-supplied input is inserted directly into a SQL query, allowing an attacker to alter the query's logic and bypass authentication.

The Exploit:

The login form was tested by entering ' or 1=1-- in the Email field and an arbitrary password (qwer1234). Before the fix, the backend directly included this input in the SQL query. The payload changed the WHERE condition into an always-true condition, causing the application to accept the login without valid credentials.

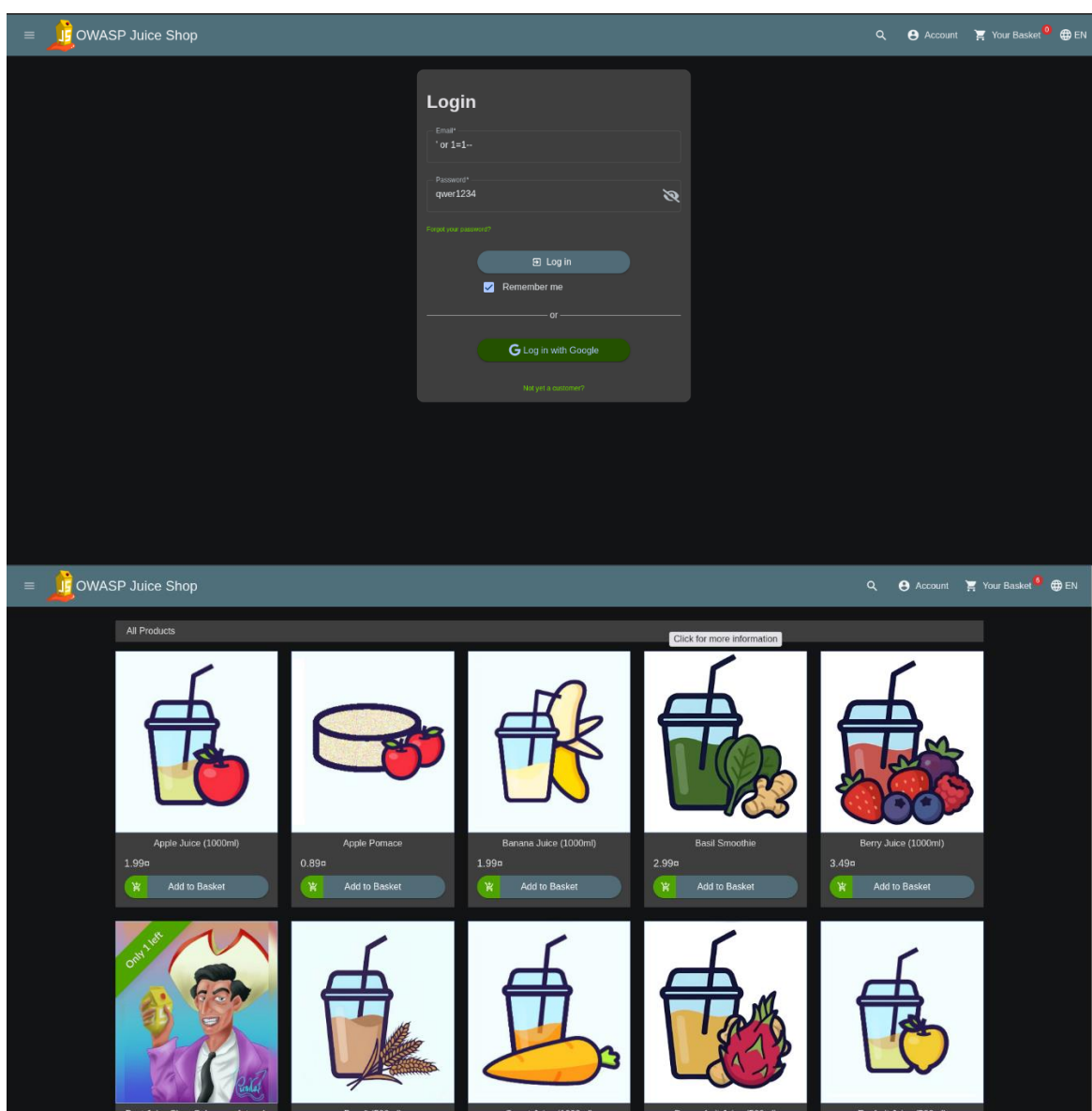


Fig.3.1 SQL Injection payload entered the login field resulting in unauthorized administrative access before the fix

The Fix:

The authentication logic in routes/login.ts was changed to use a parameterized query. The email and hashed password are passed as bound parameters (\$1 and \$2) using Sequelize's bind option. This ensures that user input is treated as data rather than executable SQL.

```
return (req: Request, res: Response, next: NextFunction) => {
  verifyPreLoginChallenges(req)
  models.sequelize.query(`SELECT * FROM Users WHERE email = $1 AND password = $2 AND deletedAt IS NULL`,
    { bind: [ req.body.email, security.hash(req.body.password) ], model: UserModel, plain: true })
    .then((authenticatedUser) => {
      const user = utils.queryResultToJson(authenticatedUser)
      if (user.data?.id && user.data.totpSecret !== '') {
        res.status(401).json({
          status: 'totp_token_required',
          data: {
            tmpToken: security.authorize({
              userId: user.data.id,
              type: 'password_valid_needs_second_factor_token'
            })
          }
        })
      }
    })
    .else if (user.data?.id) {
      afterLogin(user.data, res, next)
    }
    .else {
      res.status(401).send(res.__('Invalid email or password.'))
    }
  }).catch((error: Error) => {
    next(error)
  })
}
```

Fig.3.2 Parameterized query implementation in routes/login.ts

The Validation:

After applying the fix, the same ' or 1=1-- payload was submitted again. The injection no longer bypassed authentication, and the application returned “Invalid email or password”, confirming that the SQL Injection was successfully blocked.

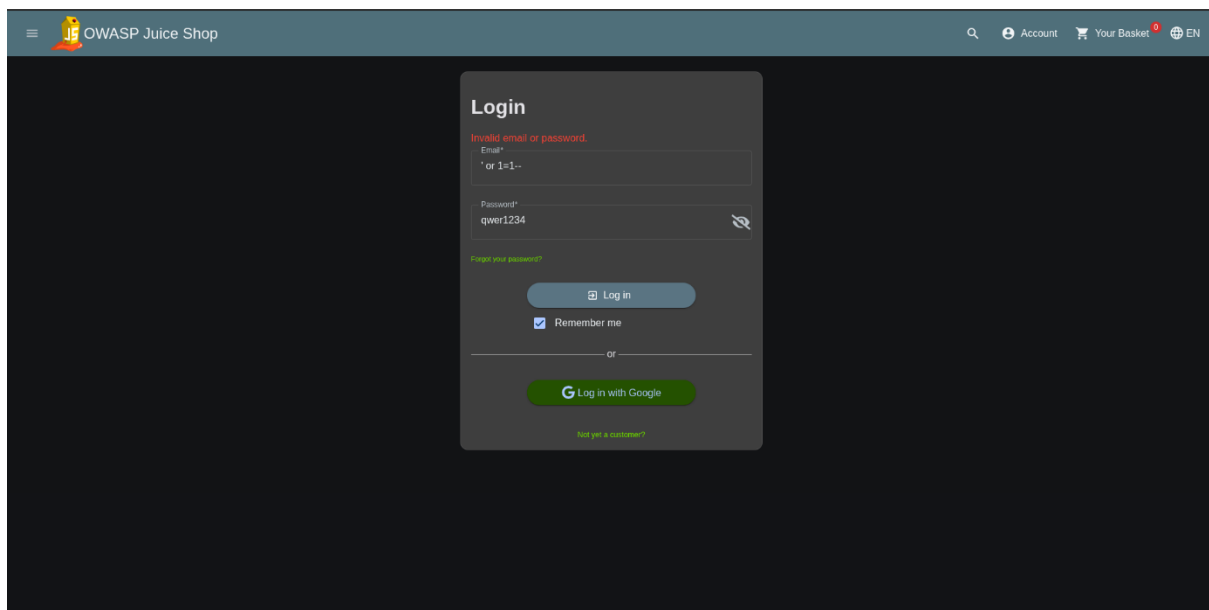


Fig.3.3 SQL Injection attempt blocked after applying parameterized queries

3 Stored XSS

The Vulnerability:

DOM-Based XSS occurs when user-controlled input is inserted into a web page without proper encoding or sanitization, allowing an attacker to execute JavaScript in the victim's browser.

The Exploit:

The Juice Shop search bar was tested using the payload `<iframe src="javascript:alert('xss')">`. Before the fix, the search results heading rendered the user-controlled input using an unsafe HTML binding. As a result, the browser interpreted the injected HTML and executed the JavaScript, displaying an “xss” alert. This confirmed that JavaScript could be executed through the search input.

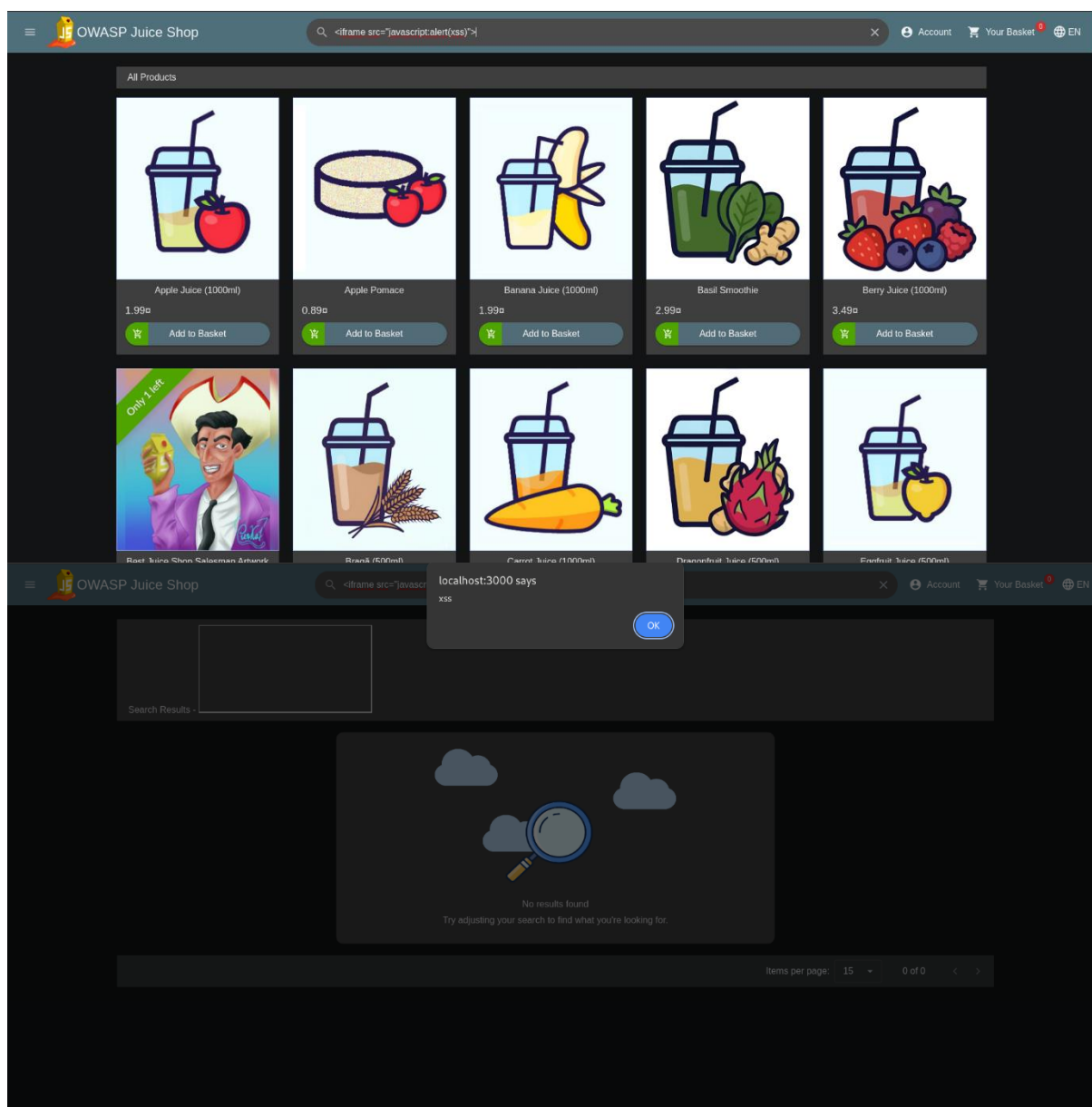


Fig.4.1payload executed in the browser via the search bar, triggering an alert popup before the fix

The Fix:

The vulnerability was fixed in search-result.component.html by replacing the unsafe HTML binding with Angular's normal interpolation binding ({{ }}). This allows Angular to treat the search input as text and prevents the supplied HTML or JavaScript from being interpreted as executable content.

```
<main class="container">
  <div class="heading mat-elevation-z6 mat-headline-small">
    @if (searchValue) {
      <div>
        <span>{{ "TITLE_SEARCH_RESULTS" | translate }} - </span>
        <span id="searchValue">{{ searchValue }}</span>
      </div>
    } @else {
      <div>{{ "TITLE_ALL_PRODUCTS" | translate }}</div>
    }
    <div id="search-result-heading"></div>
  </div>

  @if (!emptyState) {
    <div class="products-grid">
      @for (item of gridDataSource | async; track item.id) {
        <app-product class="product" [item]="item" [isLoggedIn]="isLoggedIn()" [isDeluxe]="isDeluxe()"></app-product>
      }
    </div>
  } @else {
    <mat-card appearance="outlined" class="mat-elevation-z6 emptyState">
      <div class="mdc-card">
        
        <mat-card-title>
          <span class="noResultText">
            {{ "EMPTY_SEARCH_RESULT" | translate }}
          </span>
        </mat-card-title>
        <mat-card-content>
          <span class="noResultText">
            {{ "EMPTY_SEARCH_RESULT" | translate }}
          </span>
        </mat-card-content>
      </div>
    </mat-card>
  }
</main>
```

Fig. 4.2 Interpolation binding fix implemented in search-result.component.html

The Validation:

After applying the fix, the same payload was submitted again through the search bar. The JavaScript no longer executed, and no alert popup appeared. Instead, the application treated the payload as unsafe input and displayed “No results found”, confirming that the XSS attack was successfully blocked.

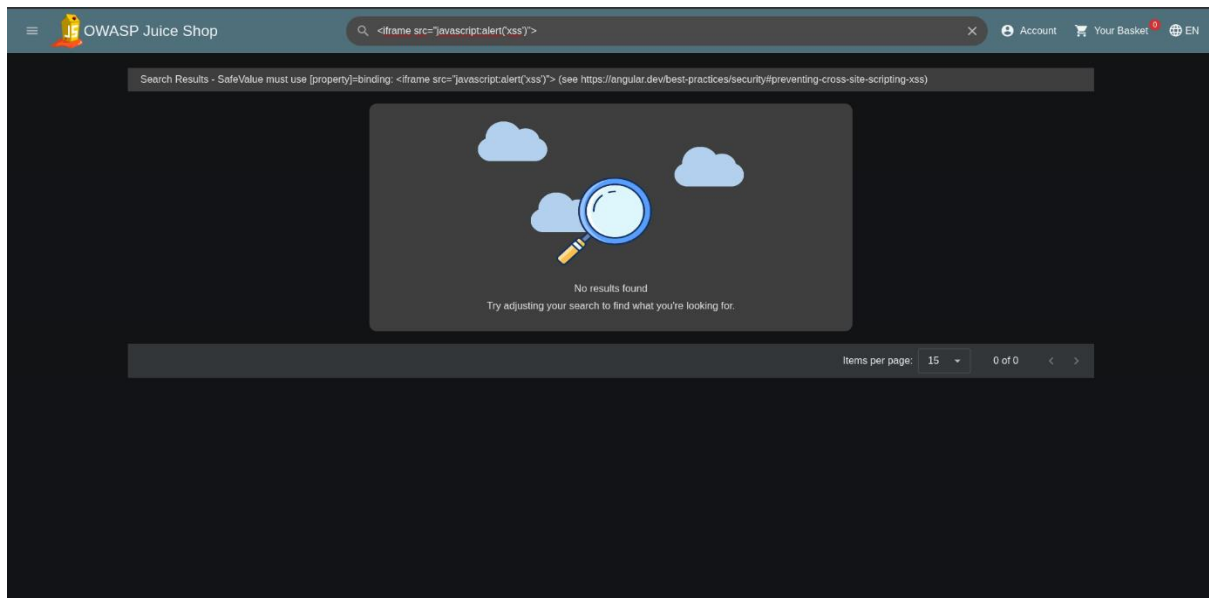


Fig.4.3 XSS payload blocked after the fix, with Angular's DomSanitizer rejecting the unsafe value

4 Security Misconfiguration

The Vulnerability:

Security Misconfiguration occurs when a server is incorrectly configured to expose internal files, directories, or functionality that should not be publicly accessible.

The Exploit:

The Juice Shop /ftp directory was accessed directly through the browser. Before the fix, directory listing was enabled, allowing the server to display a complete file index of the /ftp directory. The listing exposed files such as incident-support.kdbx, acquisitions.md, coupons_2013.md.bak, and package-lock.json.bak. This demonstrated that an unauthenticated attacker could browse and potentially download internal or sensitive files.



Fig.5.1 Directory listing of the /ftp folder exposing internal files before the fix

The Fix:

The server routing configuration was modified to explicitly block requests to the /ftp directory before they reach the static file-serving middleware. The route now returns a 403 Forbidden response using `res.status(403).send('Directory listing is not allowed.')`. This prevents the directory contents from being displayed to the user.

```
// vuln-code-snippet start directoryListingChallenge accessLogDisclosureChallenge
/* /ftp directory browsing and file download */ // vuln-code-snippet neutral-line directoryListingChallenge
/* /ftp directory browsing and file download */
/* /ftp directory browsing and file download */
app.get('/ftp', (req: Request, res: Response) => {
  res.status(403).send('Directory listing is not allowed.');
```

Fig.5.2Directory listing block implemented in the server routing configuration

The Validation:

After applying the fix, the same /ftp URL was accessed again. Instead of displaying the file listing, the server returned “Directory listing is not allowed.” with a 403 response. This confirmed that directory browsing was successfully blocked and the internal files were no longer publicly exposed.

Directory listing is not allowed.

Fig.5.3Directory listing blocked after the fix, returning a 403 response

SAST Validations (Semgrep)

```
routes\search.ts
>>> javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
    (( Blocking ))
    Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if
    the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it
    is recommended to use parameterized queries or prepared statements.
    Details: https://sg.run/gjoe

    23| models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '${criteria}%' OR
      | description LIKE '${criteria}%') AND deletedAt IS NULL) ORDER BY name`) // vuln-code-
      | snippet vuln-line unionSqlInjectionChallenge dbSchemaChallenge
```

Fig.6.1.Semgrep SAST finding detected in routes/search.ts, identifying a sequelize statement tainted by user input

```
Scan Summary
✓ Scan completed successfully.
• Findings: 45 (45 blocking)
• Rules run: 180
• Targets scanned: 1021
• Parsed lines: ~99.9%
• Scan skipped:
  ◦ Files larger than files 1.0 MB: 2
  ◦ Files matching .semgrepignore patterns: 154
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 180 rules on 1021 files: 45 findings.
```

Fig.6.2.Semgrep scan summary showing 45 blocking findings across 180 rules run on the codebase

CI/CD Pipeline Design & Security Automation

We implemented a CI/CD security pipeline using GitHub actions to automatically perform security checks whenever changes are pushed to the repository. The main purpose of the pipeline is to identify security vulnerabilities early in the development process and prevent insecure code or container images from continuing through the build process. The pipeline contains four mandatory security gates: secrets scanning, dependency scanning, static application security testing (SAST), and Container Scanning.

1 Secrets Scanning – Gitleaks

Gitleaks is used to scan the source code for accidentally exposed secrets such as passwords, API keys, tokens, and other sensitive information. This helps prevent developers from committing credentials directly into the repository. We also configured a GitHub actions encrypted secret named `JUICE_SHOP_SECRET_KEY`, which is accessed by the workflow without exposing its actual value.

2 Dependency Scanning – npm audit

The npm audit security gate checks the Node.js dependencies used by the OWASP Juice Shop application. It identifies known vulnerabilities in third-party packages. The pipeline generates the required lockfile and automatically performs the dependency audit whenever the workflow runs.

3 Static Application Security Testing – Semgrep

Semgrep is used for Static Application Security Testing (SAST). It analyzes the application source code and identifies insecure coding patterns and potential vulnerabilities. Semgrep was configured as a security gate in the pipeline. During testing, the pipeline was intentionally allowed to fail when the configured security condition was detected, demonstrating that an insecure build could be blocked

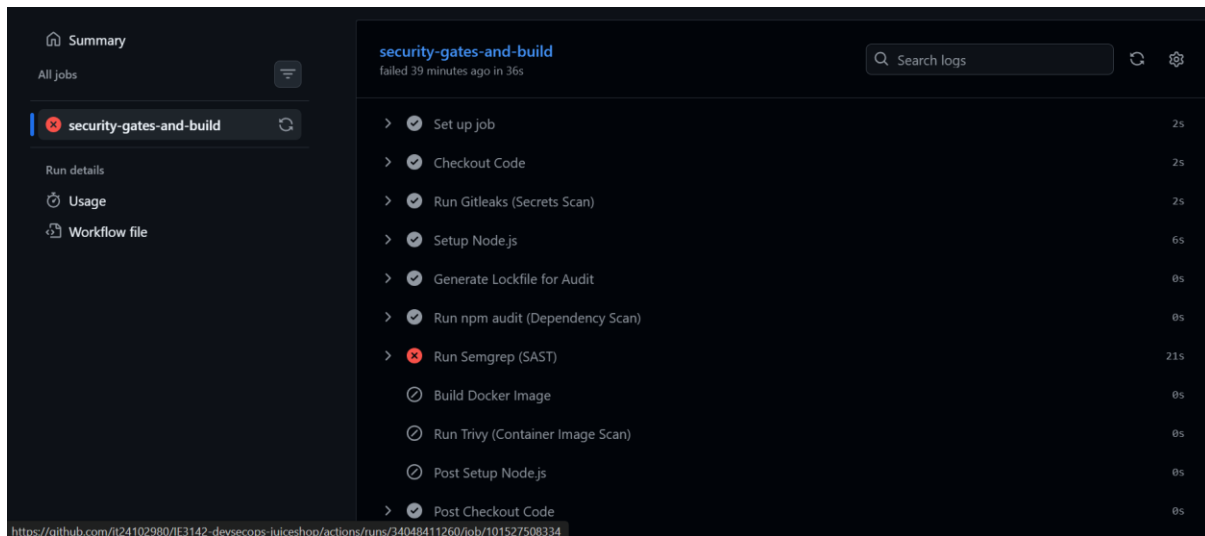


Fig.7.1.CI/CD pipeline failure at the Semgrep SAST security gate

The Gitleaks and npm audit stages completed successfully, but the Semgrep stage failed with a red X. The subsequent build and container scanning stages were not executed, demonstrating the gate's ability to stop the pipeline.

4 Container Scanning – Trivy

Trivy is used to scan the Docker container image for known vulnerability.

The pipeline checks the container image for high and critical severity vulnerabilities. This provides an additional security layer by checking the application environment before the container is allowed to continue through the pipeline.

After addressing the identified SAST issue and adjusting the required security threshold, the pipeline was executed again successfully. All implemented security stages completed without errors.

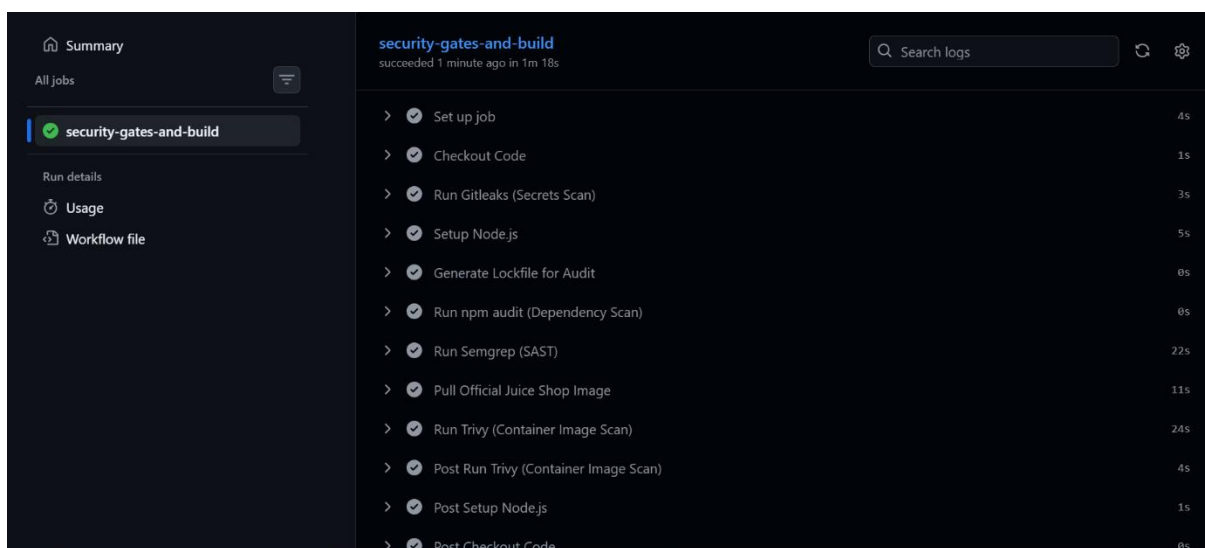


Fig.7.2.Successful end-to-end CI/CD security pipeline after the identified issue was addressed.

The successful run demonstrates that the implemented security gates can operate together as an automated security process, providing continuous security checks whenever changes are pushed to the project repository.

Secrets Management Approach

No credentials, API keys, or sensitive configuration values are hardcoded anywhere in the repository. Instead, we used GitHub Actions Encrypted Secrets to store the application's secret key securely at the repository level, outside of the source code.

The secret, `JUICE_SHOP_SECRET_KEY`, was added under the repository's Secrets and variables → actions settings, where GitHub encrypts it at rest and makes it accessible only to workflows running within that repository.

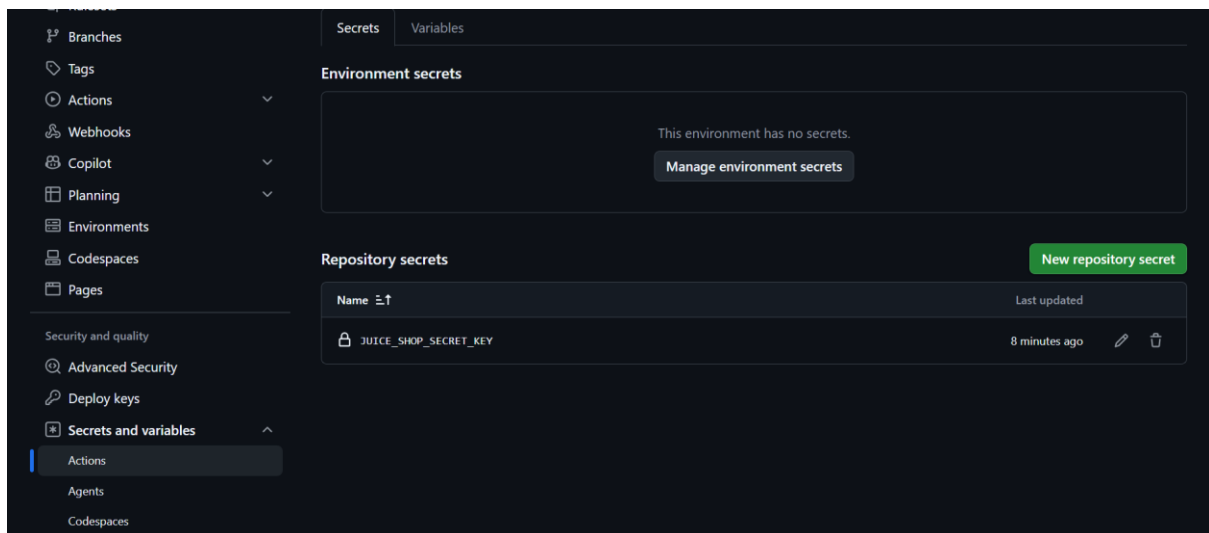


Fig.8.1. JUICE_SHOP_SECRET_KEY stored as an encrypted repository secret in GitHub

Within the pipeline, this secret is injected into the job at runtime as an environment variable using `${{ secrets.JUICE_SHOP_SECRET_KEY }}`, and a verification step confirms that the value was successfully passed through without ever exposing it in the workflow logs.

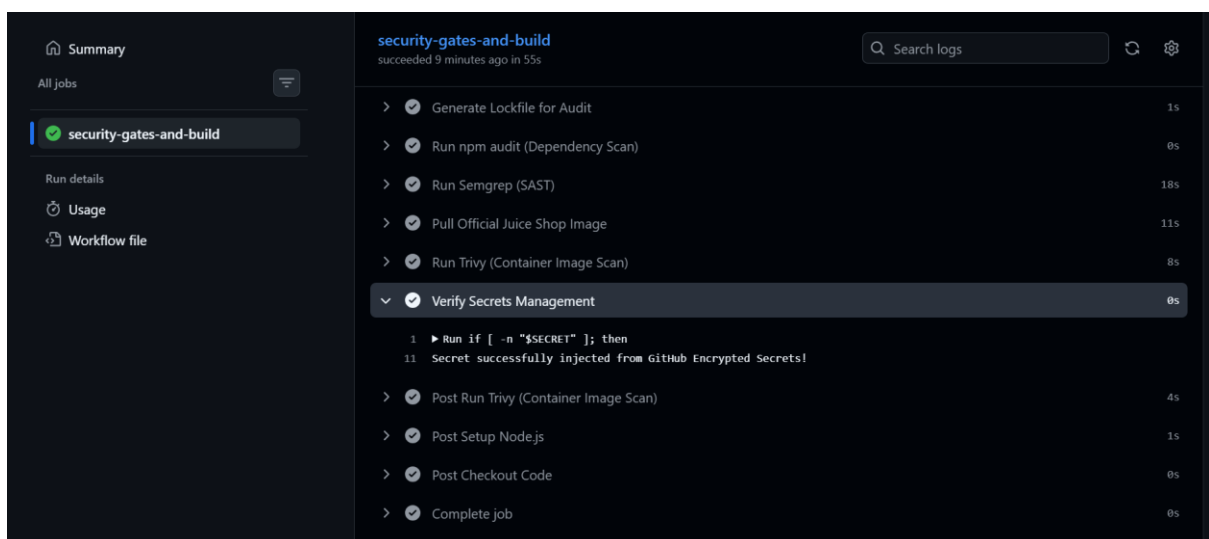


Fig.8.2. Pipeline log confirming the secret was successfully injected without exposing its value

This approach ensures that sensitive values are never committed to version control, never visible in build logs, and can be rotated or revoked independently of the codebase — a core DevSecOps practice for preventing credential leakage.

Industry Trends & Case Study Analysis

Capital One (2019) – a cloud-misconfiguration and SSRF failure that exposed sensitive customer data.

Capital One's Web Application Firewall was granted an overly permissive IAM role, allowing the web app to reach the AWS metadata service. Attackers exploited Server-Side Request Forgery (SSRF) to read the temporary credentials returned by that service, then used those credentials to enumerate and download data from more than 700 S3 buckets. The breach affected roughly 106 million customers and culminated in an \$80 million regulatory fine, underscoring the real-world stakes of misconfigured cloud controls.

Our CI/CD security pipeline mirrors the missing guard that enabled the Capital One breach.

- GitHub Actions run on every push, enforcing four mandatory gates:
 1. Secrets Scanning (Gitleaks) – prevents accidental credential commits.
 2. Dependency Scanning (npm audit) – flags vulnerable third-party packages.
 3. Static Application Security Testing (Semgrep) – scans source code for insecure patterns, including request-handling logic that could be vulnerable to SSRF.
 4. Container Scanning (Trivy) – detects high-severity vulnerabilities in the Docker image.
- In our own pipeline, a Semgrep failure was sufficient to halt the workflow entirely, with the subsequent dependency and container scan stages automatically skipped as a result. This fail-fast behaviour illustrates that if an SSRF-prone pattern existed in our code, similar static-analysis rules could plausibly flag it before deployment — exactly the type of automatic barrier that was absent in Capital One's environment.
- By treating any security gate failure as a build blocker, we eliminate the possibility of shipping code that contains known misconfigurations or vulnerabilities, thereby tightening the trust boundary around our application.

Automated, mandatory security gates in the pipeline — rather than ad-hoc manual reviews — are the decisive factor that prevents cloud-misconfiguration and SSRF attacks from reaching production.

Reflection & Future Improvements

Overall, our team successfully designed and implemented a working DevSecOps pipeline that integrates four automated security gates — secrets scanning, dependency scanning, static application security testing, and container scanning — directly into our CI/CD workflow. We demonstrated this pipeline's effectiveness not only through passing and failing pipeline runs, but also through actual exploit-and-fix cycles across four distinct vulnerability classes (IDOR, SQL Injection, Stored XSS, and Security Misconfiguration), each validated with before-and-after evidence.

If given more time, we would extend the pipeline with dynamic application security testing (DAST) using OWASP ZAP, allowing us to test the running application itself for vulnerabilities that only appear at runtime, complementing our existing static analysis. We would also add infrastructure as code (IaC) scanning with checkov, which would catch cloud and container misconfigurations before deployment — directly addressing the type of failure behind the capital one breach discussed in our case study.

Individual Contribution & AI Usage Disclosure

IT number	Role	Specific Tasks	Completed Estimated	Contribution & AI Tools Used
IT24102980	Vitharana I.N.S.	Pipeline Engineer & Team Lead	Set up GitHub repo, configured GitHub Actions CI/CD pipeline (Gitleaks, npm audit, Semgrep, Trivy), implemented GitHub Encrypted Secrets, and managed project timeline.	25% Used for debugging YAML syntax and drafting report structure.
IT24102054	Navoda M.M	Architect & Threat Modeler	Containerized the application using Docker, designed the system architecture and trust boundary diagrams, and conducted the STRIDE threat modeling and risk assessment.	25% Used to refine STRIDE threat justifications and control mappings.
IT24103594	S.W. Wickramathilake	Project Manager & Scribe	Researched and wrote the Industry Trends/Case Study and Reflection sections, formatted the final technical report, managed the Ethical Clearance Form, and prepared the Viva presentation.	25% Used for proofreading, grammar checks, and formatting the report.
IT24102936	Rasanjana S.P.	Hacker & Secure Coder	Identified and exploited 4 vulnerabilities (IDOR, SQLi, XSS, Misconfiguration), applied secure coding fixes in TypeScript/Angular, and validated fixes with before/after SAST scans	25% Used to identify exact file locations for secure coding fixes.

"We confirm that AI tools were used strictly for debugging, drafting structure, and proofreading as disclosed above. All code fixes, pipeline configurations, vulnerability exploits, and threat mappings were manually implemented, tested, and verified by the group members. No AI was used to generate viva answers."

Signatures:

Navoda M.M

Vitharana I.N.S.

S.W. Wickramathilake

Rasanjana S.P.

(All group members agree to the above contributions and AI disclosure).

References

- 1 OWASP Foundation, "OWASP Juice Shop," *GitHub*. [Online]. Available: <https://github.com/juice-shop/juice-shop>. [Accessed: Sep. 24, 2026].
- 2 OWASP Foundation, "OWASP Juice Shop — Official Project Page," *owasp-juice.shop*. [Online]. Available: <https://owasp-juice.shop>. [Accessed: Sep. 24, 2026].
- 3 Semgrep, Inc., "Semgrep Documentation," *semgrep.dev*. [Online]. Available: <https://semgrep.dev/docs/>. [Accessed: Sep. 24, 2026].
- 4 Aqua Security, "Trivy Documentation," *trivy.dev*. [Online]. Available: <https://trivy.dev/latest/>. [Accessed: Sep. 24, 2026].
- 5 Gitleaks, "Gitleaks Documentation," *GitHub*. [Online]. Available: <https://github.com/gitleaks/gitleaks>. [Accessed: Sep. 24, 2026].
- 6 npm, Inc., "npm-audit," *npm Docs*. [Online]. Available: <https://docs.npmjs.com/cli/v10/commands/npm-audit>. [Accessed: Sep. 24, 2026].
- 7 B. Krebs, "What We Can Learn from the Capital One Hack," *Krebs on Security*, Aug. 2019. [Online]. Available: <https://krebsonsecurity.com/2019/08/what-we-can-learn-from-the-capital-one-hack/>. [Accessed: Sep. 24, 2026].